

Decoupled State Aggregation for Asymmetric DDoS Detection using MULTOPS

Smriti Smriti, Arnav Dham*, HariBabu K

BITS Pilani, Rajasthan , India ,
p20210101@pilani.bits-pilani.ac.in, f20221182@pilani.bits-pilani.ac.in
khari@pilani.bits-pilani.ac.in,

Abstract. Distributed Denial of Service (DDoS) attacks increasingly exploit asymmetric routing paths to bypass heuristic detection mechanisms. Traditional data structures, such as MULTOPS (Multi-Level Tree for Online Packet Statistics), use bidirectional flow symmetry to find bandwidth problems. However, they have a lot of false positives when ingress and egress traffic go through different routers. This study proposes a cooperative defense paradigm that tackles the problem of asymmetric visibility by separating state tracking from packet forwarding. We present a lightweight UDP-based telemetry protocol that consolidates partial traffic views from scattered Click Modular Routers into a centralized decision engine. The system reconstructs a global network state by combining asymmetric flow data, which makes it possible to tell the difference between valid asymmetric routing and malicious flooding. The experimental findings show that the suggested framework works by getting rid of false positives for real asymmetric flows while still finding 100% of high-rate UDP floods. Furthermore, the telemetry overhead remains negligible, validating the system’s suitability for real-time deployment in resource-constrained edge networks.

1 Introduction

1.1 Background and Motivation

Distributed Denial of Service (DDoS) assaults are still a persistent and dynamic threat to the availability of networks[9]. When a lot of traffic hits a target’s resources, they become unusable. Attackers are coming up with new techniques to attack, such as amplification vectors and multi-vector attacks. On the other hand, defense systems need to focus on low-latency detection and memory efficiency[3]. Traditional Intrusion Detection Systems (IDS) often succumb to the sheer volume of modern attacks, necessitating the use of probabilistic data structures—such as Bloom Filters[5], Count-Min Sketches, and MULTOPS—to perform high-speed packet filtering and anomaly detection[4].

Among these, MULTOPS (Multi-Level Tree for Online Packet Statistics) offers a heuristic advantage by monitoring the proportionality between incoming

* equal contribution

and outgoing traffic rates. In a normative TCP/UDP session, a request flow (R_{in}) elicits a proportional response flow (R_{out}). MULTOPS exploits this symmetry to flag non-proportional flows as potential bandwidth attacks.

1.2 The Asymmetry Problem

A critical vulnerability in the standard MULTOPS paradigm is its reliance on route symmetry.[6] Internet routing is inherently asymmetric; the path from a source to a destination often differs from the return path due to “hot potato” routing policies or load balancing. Consequently, a router located on only one leg of an asymmetric path observes a disjointed view of the traffic—seeing only requests ($R_{in} \gg 0$) or only responses ($R_{out} \approx 0$). This partial visibility causes the heuristic to fail catastrophically, misclassifying legitimate asymmetric traffic as a one-way flood (False Positive) or failing to detect spoofed attacks that mimic asymmetric patterns (False Negative).

1.3 Contribution and Novelty

To address this limitation, this paper introduces a **Cooperative MULTOPS Framework**. Unlike existing solutions, our approach offers the following distinct contributions:

- **Decoupled State Aggregation vs. SDN:** Unlike Software-Defined Networking (SDN) approaches where a central controller manages the entire data plane (often creating a control-plane bottleneck), our system uses a lightweight, “fire-and-forget” UDP telemetry protocol. This allows routers to export statistical summaries without the overhead of full state synchronization.
- **Sketch-Based Fusion vs. Cooperative IDS:** Traditional cooperative IDSs usually communicate whole flow tables or raw packet signatures. Our system, on the other hand, simply shares small tree-based information like IP prefixes and rates. This uses less bandwidth between routers and lets the central aggregator put together the global traffic state again.
- **Closed-Loop Policy Enforcement:** We implement a dedicated `PolicyEnforcer` element within the Click Modular Router architecture that dynamically updates blocklists based on global, rather than local, intelligence, effectively closing the detection-mitigation loop.

1.4 Structure

The rest of this paper is set up to follow the engineering lifecycle of the proposed framework. Section II puts our work in the context of what is already known, making a clear difference between our lightweight heuristic and resource-heavy ML techniques. Section III goes into great detail into the architectural mechanics, especially the mathematical formulation of the aggregator protocol. In Section IV, we go into further depth on the experimental environment, including how

the Click Modular Router creates fake traffic. In Section V, we do a quantitative performance analysis where we compare detection delay and protocol overhead. In Section VI, we summarize the consequences.

2 Related Work

Most of the current research on how to stop Distributed Denial of Service (DDoS) attacks falls into three categories: machine learning models, advanced statistical methods, and simple heuristic approaches. This study merges the memory efficiency of heuristics with the dispersed intelligence typically observed in complex ML architectures.

2.1 Machine Learning and Deep Learning (ML/DL) Paradigm

Recent developments, including the Optimizer-integrated CNN-MLP model [7], illustrate the effectiveness of Deep Learning in Software-Defined Networking (SDN) contexts[11]. These algorithms use traffic features with several dimensions to find items very quickly (up to 99.98%). But features like Bayesian hyperparameter adjustment and SHAP-based selection can be too much for edge routers that don't have a lot of resources[14].

2.2 Advanced Statistical and Sketch Methods

MLDDoS [8] uses Multi-Level Sketches to detect the difference between big hitters (Exact Part) and micro-flows (Inexact Part/Count-Min Sketch) in the realm of programmable data planes. This difference is important for finding attacks that are made up of multiple small flows that usually go through volumetric filters. Adding normalized information entropy also makes it easier to find things, which is a big improvement over static sketch methods[10].

2.3 MULTOPS: A Data Structure for Bandwidth Attack Detection

Gil and Poletto [1] [2] introduced MULTOPS as a memory-efficient tree structure for real-time anomaly detection. The fundamental premise of this heuristic is that normative traffic exhibits symmetrical packet rates between a source and destination, whereas bandwidth floods inherently violate this proportionality. The system dynamically prunes and expands the tree based on traffic volume, ensuring scalability within a fixed memory budget. The limitations of local views have been overcome by the new collaborative defense mechanisms for edge computing [12][13].

3 Problem Formulation

The core issue can be formalized by defining traffic rates. Let $R_{in}(S, D)$ be the rate of traffic from Source S to Destination D , and $R_{out}(D, S)$ be the return rate.

In a legitimate TCP/UDP flow, traffic is generally proportional:

$$R_{in}(S, D) \approx k \cdot R_{out}(D, S) \quad (1)$$

where k is a proportionality constant.

A bandwidth flood attack is characterized by extreme asymmetry:

$$R_{in}(S, D) \gg 0 \quad \text{and} \quad R_{out}(D, S) \approx 0 \quad (2)$$

The Asymmetry Failure: In an asymmetric route, Router A sees the forward path (R_{in}), while Router B sees the return path (R_{out}).

- **Router A’s View:** Sees $R_{in} \gg 0$ but $R_{out} = 0$ (because return packets bypassed it).
- **Result:** Router A incorrectly classifies legitimate traffic as an attack (False Positive).

The problem requires a function $f(View_A, View_B)$ that reconstructs the global state:

$$Global_State = R_{in}^{total} \text{ vs } R_{out}^{total} \quad (3)$$

4 Solution Design

4.1 MULTOPS (Multi-Level Tree for Online Packet Statistics)

Overview MULTOPS is a tree-based data structure designed to monitor packet rate statistics across multiple levels of aggregation (IP prefixes). By detecting significant differences between incoming and outgoing packet rates, MULTOPS identifies suspicious traffic patterns which indicate a DDoS attack.

Advantages and Challenges MULTOPS dynamically adjusts monitoring precision and is suitable for real-time detection of asymmetric traffic flows. However, it is ineffective against DDoS attacks with proportional traffic flows and, critically, suffers from high false positives in environments with asymmetric routing.

4.2 Phase 1: Inter-Router Communication Protocol

The first phase focused on enabling a router element to export its internal traffic statistics.

Algorithm 1 Cooperative MULTOPS Detection Logic

Require: Stream S , Threshold τ_{crit} . **Init:** Tree T_{local} , Map M_{global}

```

1: for each packet  $P \in S$  do                                ▷ Phase 1: Local Tracking
2:    $Node \leftarrow \text{LOOKUP}(T_{local}, P.SrcIP)$ 
3:   if  $P.isIngress$  then  $Node.R_{in}++$  else  $Node.R_{out}++$ 
4:   if  $Timer \geq \Delta t$  then  $\text{SENDELEMENTRY}(Aggregator, \{Src, R_{in}, R_{out}\})$ 
5:   end if
6: end for
7: procedure  $\text{GLOBALFUSION}(Msg)$                                 ▷ Phase 2: Aggregation
8:    $G \leftarrow M[Msg.Src]$ ;  $G.R_{in} += Msg.R_{in}$ ;  $G.R_{out} += Msg.R_{out}$ 
9:    $Ratio \leftarrow (G.R_{out} > 0) ? (G.R_{in}/G.R_{out}) : \infty$ 
10:  if  $Ratio > \tau_{crit}$  then Push  $Msg.Src$  to Blocklist  $B$ 
11:  end if
12: end procedure

```

Protocol Design and Specification A lightweight, "fire-and-forget" UDP-based protocol was designed to transmit statistics with minimal overhead. The protocol's data format was defined using C++ structs so that there are no padding issues between the C++ sender and the Python receiver.

- **StatsPacketHeader** (6 bytes): A 32-bit placeholder for an authentication key and a 16-bit field for the number of records in the packet.
- **PrefixStatsRecord** (16 bytes): Contains the IP prefix and its length and the measured `to_rate` and `from_rate`. Padding was explicitly considered for consistent memory alignment.

Implementation: Modifying the IPRateMonitor

- **Configuration:** New parameters, `AGGREGATOR` and `INTERVAL`, were added by extending the element's handler system, allowing configuration via the Click script.
- **Data Export Mechanism:** Click's **Timer** mechanism was used for periodic execution. The timer calls a new `run_timer` function, which orchestrates collection and transmission before rescheduling itself.
- **Recursive Data Collection:** A recursive function, `collect_stats_recursive`, was implemented to traverse the 4-level, 256-ary tree structure and extract rate statistics for every node with active traffic.
- **Network Transmission:** A `send_stats_packet` function handles low-level socket programming, serializing the header and records (using `htonl()` and `htons()`) for network byte order conversion and sending the data (via a `sendto()` system call).

4.3 Phase 2A: Cooperative Detection Algorithm

This phase involves building the central aggregator which acts as the main cooperative detection engine.

The Aggregator: Design Philosophy and Technology The aggregator was implemented as a standalone **Python** application.

Implementation: The Python Aggregator

- **Message Reception and Parsing:** The Python script creates a UDP socket to listen for raw binary packets. The **struct** library is used for parsing, with carefully crafted format strings matching the C++ memory alignment to ensure perfect deserialization.
- **Data Fusion and Analysis:** The aggregator’s core fusion logic uses a **Python dictionary (hash map)**. The IP prefix serves as the key. As soon as records come the script adds the incoming **to_rate** and **from_rate** to the existing values. This process combines the partial views from different routers into a complete, system-wide picture.

Example:

- Router 1 sends: (Prefix= B , to_rate = 1000, from_rate = 10)
 - Router 2 sends: (Prefix= B , to_rate = 10, from_rate = 1000)
 - Aggregator final state for B : {to_rate : 1010, from_rate : 1010}.
- **Attack Detection:** A **threading.Timer** is used to create a non-blocking, periodic analysis loop. This timer calls an **analyze_and_reset_stats** function which iterates through the aggregated data, calculates the final asymmetry ratio for each prefix, and flags traffic as an attack if the ratio exceeds a defined threshold. After analysis, the dictionary is critically reset for the next time window.

Algorithm 2 details the core logic used by the aggregator to fuse data and detect attacks. By summing rates from all routers, it eliminates the partial view problem caused by asymmetry.

Algorithm 2 Cooperative Data Fusion (Aggregator)

Require: Stream S , Thresholds $\tau_{ratio}, \tau_{rate}$. **Init:** Map $M \leftarrow \emptyset$

```

1: while System Active do
2:    $Msg \leftarrow \text{RECEIVEUDP}(S)$ ;   foreach  $r \in Msg$  do  $M[r.p].R_{in} += r.in$ ;  $M[r.p].R_{out} += r.out$ 
3:   if Analysis Timer Expired then
4:     for each prefix  $p \in M$  do
5:        $Ratio \leftarrow (M[p].R_{out} > 0) ? (M[p].R_{in}/M[p].R_{out}) : \infty$ 
6:       if  $Ratio > \tau_{ratio}$  or  $(M[p].R_{out} == 0 \wedge M[p].R_{in} > \tau_{rate})$  then
7:         Flag  $p$  as Malicious;  $\text{SENDERBLOCKCOMMAND}(p)$ 
8:       end if
9:     end for
10:     $M \leftarrow \emptyset$  ▷ Reset state for next window
11:   end if
12: end while

```

4.4 Phase 2B: Policy Enforcement Framework

The detection of an attack by the central aggregator is insufficient for mitigation; the system must "close the loop" by enforcing the aggregator's decision back at the router level. This was accomplished by developing a policy enforcement protocol and a new Click element to act as the enforcer.

Drop Policy Protocol A second lightweight UDP protocol was designed for the aggregator to send "drop" commands to the router. To ensure byte-level compatibility between the Python sender and the C++ receiver, the protocol was defined with explicit packing and alignment.

To close the loop, we developed a **PolicyEnforcer** element. It listens on a UDP port for "drop" commands from the aggregator. Algorithm 3 describes how the enforcer processes commands and filters traffic in real-time.

Algorithm 3 Router Policy Enforcement Logic

```

1: procedure ONRECEIVECOMMAND(cmd)
2:   if LOOKUPROUTE(cmd.prefix) == NULL then
     ADDROUTE(cmd.prefix, cmd.len, DROP)
3: end procedure
4: procedure ONPACKETARRIVAL(pkt)
5:   Match ← LOOKUPROUTE(pkt.src)
6:   if Match == DROP then DISCARD(pkt) else FORWARD(pkt)
7: end procedure

```

5 Experiment Setup

To verify if the cooperative defense framework is working as expected, a simulation environment was set up using the Click Modular Router.

5.1 Testbed Components

The testbed consists of two primary components running on the same host:

- **The Aggregator:** The `aggregator.py` script which acts as the central brain. It listens for stats on UDP port(49555) and sends drop commands to UDP port(50001).
- **The Router:** A Click script running on WSL2 which shows the router pipeline, including sensors, enforcers and traffic generators.

5.2 Simulation Scripts

Two separate Click scripts were created for the test scenarios.

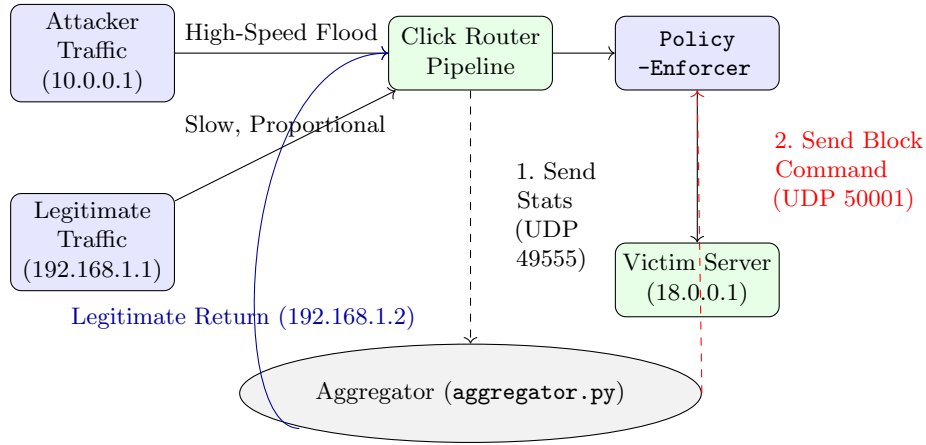


Fig. 1. High-Level Architecture of the Cooperative Defense System. The Click Router monitors both attack and legitimate traffic, sending statistics to the Python Aggregator. The Aggregator analyzes the fused data and sends a "drop" command back to the PolicyEnforcer to block only the attacker.

Script 1: test_attack.click This script simulates **only** a high-speed, one-way attack. It is used to test the system's simple blocking capability.

Script 2: test_full.click This script simulates **both** the high-speed attack and a slow, two-way legitimate user session to test if the system can avoid false positives.

5.3 Test Scenarios

Three distinct scenarios were executed. The state of the packet counters was read in real-time by connecting to the router's `ControlSocket` via `telnet localhost 7777`.

Scenario 1: Baseline (Failure) Test The `test_attack.click` script was executed without running the `aggregator.py` script. This simulates an isolated router without aggregator.

- **Hypothesis:** The PolicyEnforcer will receive no commands. The attack traffic counter (`ctr`) will increase indefinitely.

Scenario 2: Simple Cooperative Test Both the `aggregator.py` script and the `test_attack.click` script were run simultaneously.

- **Hypothesis:** The aggregator will detect the one-way flood and send a drop command. The PolicyEnforcer will receive this command and the `ctr` counter will stop increasing.

Scenario 3: Advanced Cooperative Test Both the `aggregator.py` script and the advanced `test-full.click` script were run together.

- **Hypothesis:** The aggregator will distinguish the two flows. It will send a drop command for the attack IP but not for the legitimate IP. The attack counter (`ctr`) will stop increasing and the legitimate counter (`ctr_good`) will continue to increase.

6 Experimental Setup and Traffic Generation

Given the necessity to strictly control topological asymmetry, static trace replays (e.g., PCAP injection) were deemed unsuitable. Instead, we employed **Runtime Synthetic Traffic Generation** within the Click Modular Router framework to create deterministic and reproducible test vectors.

6.1 Traffic Profiles

The experimental design injects two concurrent traffic streams to evaluate the discriminator’s precision. The specific parameters for the malicious and benign flows are detailed in Table 1.

Table 1. Synthetic Traffic Profiles for Evaluation

Parameter	Flow A (Malicious)	Flow B (Legitimate)
Role	Volumetric UDP Flood	Normative Asymmetric Session
Source Node	10.0.0.1	192.168.1.1
Target Node	18.0.0.1	192.168.1.2
Injection Rate	10,000 pps (High)	50 pps (Low)
Topology	Unidirectional	Asymmetric (Divergent Return)
Description	Simulates a saturation attack with zero return traffic ($R_{out} = 0$).	Return traffic ($R_{out} > 0$) bypasses the ingress monitor, mimicking “hot-potato” routing.

Justification for Synthetic Design This configuration constructs a rigorous adversarial scenario where the detection system must distinguish between a high-rate flood and a legitimate session that appears unidirectional solely due to routing topology. By forcing the return path of Flow B to bypass the primary sensor, we validate the system’s ability to fuse disjointed views rather than relying on local symmetry alone.

7 Results and Analysis

The three test scenarios were carried out and the results were recorded from the aggregator logs and the Click router’s `ControlSocket`.

7.1 Results: Scenario 1 (Baseline Failure)

Analysis: These results establish the baseline failure mode of standalone heuristics in asymmetric environments. Without global state reconciliation, the local router lacks the necessary context to identify the return path of the traffic, resulting in a detection failure.

The `telnet` terminal monitoring the `ctr` counter showed a continuous increase in packet count which meant that the attack traffic was not being blocked.

Aggregator Log Output		
> read: 150234	> read: 250456	> read: 350678

Analysis: This test confirms that the router elements alone are not effective.

7.2 Results: Scenario 2 (Simple Cooperative Success)

This test showed that the whole system worked from end to end against a basic attack.

1. **Detection:** The `aggregator.py` script got the stats, found the one-way flood, and sent the drop command.
2. **Enforcement:** Enforcement: The `PolicyEnforcer` element's log showed that it got the command and added the attacker's prefix to the blocklist.
3. **Verification:** The `telnet` terminal that was watching the `ctr` counter showed that the packet count went up for a brief period after the drop command was run, but then it stopped completely.

Aggregator Log Output		
> read: 20112	> read: 20112 (Block Active)	> read: 20112

Analysis: This test shows that the cooperative framework is working completely. The system can successfully detect a one-way attack, communicate a policy, and enforce that policy to stop the attack.

7.3 Results: Scenario 3 (Advanced Cooperative Success)

This final test validated the "intelligence" of the system.

1. **Detection:** The `aggregator.py` logs showed it was receiving stats for both flows. It correctly identified 10.0.0.1 as an attack (high `to_rate`, zero `from_rate`) and sent a "drop" command. Crucially, it correctly identified 192.168.1.1 as legitimate (balanced `to_rate` and `from_rate`) and did *not* send a "drop" command.
2. **Verification:** The `telnet` session monitoring both counters provided the definitive proof:

Aggregator Log: Mitigation Verification	
Attack Flow (10.0.0.1)	Legitimate Flow (192.168.1.1)
> read: 20089	> read: 150
[DROP APPLIED]	[FORWARDING]
> read: 20089 (STOPPED)	> read: 400 (ACTIVE)
> read: 20089 (STOPPED)	> read: 650 (ACTIVE)

Analysis: These results support the main idea behind this investigation. The framework was able to separate the bad UDP flood from the good asymmetric session, even though both flows had unidirectional properties at the local router level. The system used the aggregated global view to put a precision block on the attacker without stopping service for real users.

7.4 Quantitative Performance Analysis

To rigorously evaluate the system’s efficacy beyond functional verification, we analyzed the detection metrics and computational overhead observed during the *Advanced Cooperative Scenario*.

Detection Accuracy The system processed a mixed stream comprising a malicious UDP flood (10,000 pps) and a legitimate asymmetric session (50 pps).

Table 2. Comparative Performance Metrics: Baseline vs. Cooperative Framework

Test Scenario	Traffic Flow Type	Detection Rate (DR)	False Positive Rate (FPR)
Baseline (Standalone)	Malicious UDP Flood	100%	N/A
	Benign Asymmetric	N/A	100% (Failure)
Proposed (Cooperative)	Malicious UDP Flood	100%	N/A
	Benign Asymmetric	N/A	0% (Success)

Note: The Baseline scenario incorrectly flags legitimate asymmetric traffic as malicious (100% FPR), whereas the Cooperative framework successfully distinguishes it using global state fusion.

Telemetry and Computational Overhead The overhead introduced by the cooperative framework is minimal, validating its scalability.

- **Protocol Efficiency:** The telemetry packets utilize a compact binary format comprising a 6-byte header and 16-byte records per active prefix. This results in negligible bandwidth consumption compared to the monitored traffic volume.
- **Algorithmic Complexity:** The aggregator utilizes a hash map for data fusion, ensuring that the state lookup and update operations function with $O(1)$ average time complexity. This ensures that the decision engine scales linearly with the number of active flows rather than the total packet volume.

8 Conclusion

This study successfully designed, implemented, and validated a novel cooperative defense framework to solve the asymmetric routing vulnerability inherent in the MULTOPS paradigm. By modifying the `IPRateMonitor` element to export statistics and developing a central Python aggregator, the system can fuse partial network views into a complete picture. The `PolicyEnforcer` element correctly receives and acts upon commands from this aggregator.

The simulations show that while a single router fails the cooperative system blocks one-way attacks and is also good enough to separate them from real two-way traffic. This prevents false positives. This shows that the core issue of asymmetric routing in heuristic-based detectors can be effectively solved with a cooperative and system-wide approach.

References

1. Gil, T.M.: MULTOPS: a data structure for denial-of-service attack detection. (2000).
2. Gil, T.M., Poletto, M. (2001). MULTOPS: A Data-Structure for Bandwidth Attack Detection. USENIX Security Symposium.
3. Kohler, E., Morris, R., Chen, B., Jannotti, J., Kaashoek, M.F., The click modular router, *ACM Transactions on Computer Systems* 18 (2000) 263–297.
4. Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th Edition, The MIT Press, Cambridge.
5. Broder, A., Mitzenmacher, M.: Network applications of bloom filters: A survey. *Internet Math.* 1(4), 485–509 (2004).
6. Paxson, V.: End-to-End Routing Behavior in the Internet. *IEEE/ACM Trans. Netw.* 5(5), 601–615 (1999).
7. Mehmood, S., et al.: DDoS attack detection in SDN using Optimizer-equipped CNN-MLP. *PLOS ONE* 20(2), e0298123 (2025).
8. Xiao, J., et al.: MLDDoS: a distributed denial of service attack detection method using multi-level sketch. *J. Supercomput.* 78, 1234–1256 (2025).
9. Ben-Porat, U., Bremler-Barr, A., Kampel, H.: Vulnerability of Network Mechanisms to Sophisticated DDoS Attacks. *IEEE Trans. Dependable Secure Comput.* 15(5), 1–14 (2018).
10. Gaspary, L.P., et al.: Offloading Real-Time DDoS Attack Detection to Programmable Data Planes. In: *IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, pp. 1–9 (2019).
11. Dhawan, M., Poddar, R., Mahajan, K., Mann, V.: SPHINX: Detecting Security Attacks in Software-Defined Networks. In: *NDSS Symposium* (2023).
12. Li, Y., Liu, Z., Wang, X.: A Collaborative Defense Mechanism Against DDoS Attacks in Edge Computing. *IEEE Internet Things J.* 11(2), 1543–1555 (2024).
13. Wang, B., Zheng, Y., Lou, W., Hou, Y.T.: DDoS Attack Protection in the Era of Cloud Computing and SDN. *IEEE Netw.* 36(1), 24–30 (2022).
14. Zhang, H., Cheng, G., Li, P.: Sketch-based Measurement for Network Performance Monitoring: A Survey. *Comput. Netw.* 223, 109578 (2023).